Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

G1

– CanTho, August 2026 –

Table of Contents

I. Record of Changes 3
II. Project Management Plan 4
1. Overview 4
1.1 Scope & Estimation 4
1.2 Project Objectives 12
1.3 Project Risks 13
2. Management Approach 15
2.1 Project Process 15
2.2 Quality Management 17
2.3 Training Plan 17
3. Project Deliverables 18
4. Responsibility Assignments 19
5. Project Communications 20
6. Configuration Management 21
6.1 Document Management 21
6.2 Source Code Management 23
6.3 Tools & Infrastructures 24

I. Record of Changes

Date	A* M, D	In charge	Change Description
27/08/2026	A	TaiNH	Add 6. Configuration Management 6.1 Document Management
27/08/2026	A	MinhTH	Add 2. Management Approach
28/08/2026	A	TaiNH	Add 6.2 Source Code Management 6.3 Tools & Infrastructures
28/08/2026	A	VinhNT	Add 1. Overview 1.1 Scope & Estimation 1.2 Project Objective 1.3 Project Risk
28/8/2026	A	TanMN	Added Section 5 – Project Communications
28/8/2026	A	KhiemNPG	Add Project deliverables Add Responsibility Assignment
29/08/2026	M	VinhNT	Rename the project from EventShield to **ConcertShield System**.

*A - Added M - Modified D - Deleted

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

#	WBS Item	Complexity	Est. Effort (man-days)
1	***Reports***		*74*
1.1	Report 1	Simple	4
1.2	Report 2	Medium	7
1.3	Report 3	Complex	21
1,4	Report 4	Complex	21
1,5	Report 5	Medium	14
1.6	Report 6	Medium	7
1.7	Report 7	Medium	7
2	***Feature 1: Authentication & Account Management***		*7*
2.1	***Register***	***Medium***	2
2.2	***Login***	***Medium***	2
2.3	***Logout***	***Simple***	1
2.4	***Manage Profile***	***Medium***	1
2.5	***Update Account Information***	***Simple***	1
2.6	***Reset Password***	***Medium***	1

2.7	***Forgot Password***	***Medium***	1
3	***Feature 2: eKYC & Identity Verification Flow***		***14***
3.1	***CCCD OCR Scanning***	***Complex***	4
3.2	***Facial Image Capture***	***Medium***	2
3.3	***Face Matching***	***Complex***	3
3.4	***Liveness Detection***	***Complex***	3
3.5	***Verification Status Management***	***Medium***	2
4	***Feature 3: Role-Based Access Control (RBAC)***		***6***
4.1	***Customer Permission Management***	***Medium***	1
4.2	***Event Organizer Permission Management***	***Medium***	1
4.3	***Staff Permission Management***	***Medium***	1
4.4	***Admin Permission Management***	***Medium***	1
4.5	***Role-based Routing***	***Medium***	2
5	***Feature 4: Event Discovery & Search***		***6***
5.1	***View Event List***	***Simple***	1
5.2	***Search Events***	***Medium***	1
5.3	***Filter Events***	***Medium***	1
5.4	***View Event Details***	***Medium***	1
5.5	***View Seating Chart***	***Medium***	1
5.6	***View Ticket Tiers & Availability***	***Simple***	1

6	***Feature 5: Virtual Queue & Waiting Room***		***10***
6.1	***Join Virtual Queue***	***Medium***	2
6.2	***Display Queue Position***	***Medium***	2
6.3	***Display Estimated Waiting Time***	***Medium***	2
6.4	***Real-time Queue Updates***	***Complex***	2
6.5	***Redirect to Ticket Purchase***	***Medium***	1
6.6	***Time-limited Purchase Session***	***Medium***	1
7	***Feature 6: Interactive Seating Chart & Ticket Selection***		***12***
7.1	***Display Interactive Seating Chart***	***Medium***	2
7.2	***Display Seat Availability***	***Medium***	2
7.3	***Select Seats***	***Medium***	2
7.4	***Hold Selected Seats***	***Complex***	2
7.5	***Real-time Seat Status Synchronization***	***Complex***	2
7.6	***Prevent Double-booking***	***Complex***	2
8	***Feature 7: Dynamic QR Code & Ticket Wallet***		***6***
8.1	***View Ticket Wallet***	***Simple***	1
8.2	***View Ticket Details***	***Simple***	1
8.3	***Generate Dynamic QR Code***	***Medium***	2
8.4	***Refresh QR Code***	***Medium***	1
8.5	***Display Ticket Status***	***Simple***	1

9	***Feature 8: Ticket Return & Re-Issue Workflow***		***6***
9.1	***View Return Policy***	***Simple***	1
9.2	***Submit Ticket Return Request***	***Medium***	2
9.3	***Track Return Request Status***	***Medium***	1
9.4	***Process Refund***	***Medium***	1
9.5	***Ticket Re-Issue***	***Medium***	1
10	***Feature 9: Event Creation & Lifecycle Management***		***7***
10.1	***Create Event***	***Medium***	2
10.2	***Edit Event***	***Medium***	1
10.3	***Configure Event Schedule & Venue***	***Medium***	1
10.4	***Upload Event Images/Banners***	***Simple***	1
10.5	***Manage Event Status***	***Medium***	1
10.6	***Submit Event for Approval***	***Simple***	1
11	***Feature 10: Seating Chart Builder & Ticket Class Configuration***		***12***
11.1	***Create Seating Zones***	***Complex***	3
11.2	***Configure Seat Quantity***	***Medium***	2
11.3	***Create Ticket Classes***	***Medium***	2
11.4	***Configure Ticket Pricing***	***Medium***	2
11.5	***Configure Dynamic Pricing Rules***	***Complex***	2
11.6	***Configure Refund Policies***	***Simple***	1

12	***Feature 11: Revenue Analytics & Sales Dashboard***		***6***
12.1	***Display Tickets Sold***	***Medium***	1
12.2	***Display Revenue***	***Medium***	1
12.3	***Display Occupancy Rate***	***Medium***	1
12.4	***Display Purchasing Trends***	***Medium***	2
12.5	***Sales Dashboard***	***Medium***	1
13	***Feature 12: Real-time Check-in Monitoring***		***5***
13.1	***Display Total Scanned Tickets***	***Simple***	1
13.2	***Display Checked-in Attendees***	***Simple***	1
13.3	***Display Check-in Ratio***	***Medium***	1
13.4	***Display Hourly Gate Entry Flow***	***Medium***	1
13.5	***Real-time Check-in Updates***	***Medium***	1
14	***Feature 13: QR Code Scanner & Check-in Interface***		***8***
14.1	***QR Code Scanning***	***Medium***	2
14.2	***Validate Ticket***	***Medium***	2
14.3	***Display Validation Result***	***Simple***	1
14.4	***Detect Used Ticket***	***Medium***	1
14.5	***Detect Incorrect Event***	***Medium***	1
14.6	***Detect Counterfeit Ticket***	***Complex***	1
15	***Feature 14: Incident Reporting & On-site Support***		***7***

15.1	***Report Damaged Ticket***	***Simple***	1
15.2	***Report Lost Ticket***	***Simple***	1
15.3	***Manual Ticket Lookup***	***Medium***	2
15.4	***Search by Phone Number***	***Simple***	1
15.5	***Search by CCCD***	***Medium***	1
15.6	***Synchronize Incident Data***	***Medium***	1
16	***Feature 15: Event Approval & Dispute Resolution***		***7***
16.1	***View Pending Events***	***Simple***	1
16.2	***Approve Event***	***Medium***	1
16.3	***Reject Event***	***Medium***	1
16.4	***Manage Ticket Disputes***	***Complex***	2
16.5	***Handle Check-in Complaints***	***Medium***	1
16.6	***Handle Lost/Damaged Ticket Cases***	***Medium***	1
17	***Feature 16: AI Risk Scoring & Fraud Alert Dashboard***		***12***
17.1	***AI Risk Score Calculation***	***Complex***	3
17.2	***Ticket-scalping Bot Detection***	***Complex***	3
17.3	***Detect Abnormal Purchasing Behavior***	***Complex***	2
17.4	***Display Fraud Alerts***	***Medium***	2
17.5	***View Alert Details***	***Medium***	1
17.6	***Block Account or Ticket***	***Medium***	1

18	***Feature 17: AI Rule Configuration & System Parameters***		***7***
18.1	***Configure Ticket Purchase Limit***	***Medium***	2
18.2	***Configure Risk Score Threshold***	***Medium***	2
18.3	***Configure eKYC Trigger Rules***	***Medium***	1
18.4	***Manage AI Parameters***	***Medium***	1
18.5	***Manage Payment Gateway Parameters***	***Medium***	1
19	***Feature 18: System Activity Log & Audit Trail***		***8***
19.1	***Record System Activities***	***Medium***	2
19.2	***Record Administrative Actions***	***Medium***	1
19.3	***Record Permission Changes***	***Medium***	1
19.4	***Record Security-sensitive Operations***	***Medium***	1
19.5	***View Activity Logs***	***Medium***	1
19.6	***Filter Activity Logs***	***Simple***	1
19.7	***View Log Details***	***Simple***	1
20	***Feature 19: Real-time Notification Management***		***6***
20.1	***Queue Position Notification***	***Medium***	1
20.2	***Payment Success Notification***	***Simple***	1
20.3	***Upcoming Event Reminder***	***Medium***	1
20.4	***Ticket Expiration Notification***	***Medium***	1
20.5	***Check-in Success Notification***	***Simple***	1

20.6	***View & Manage Notifications***	***Simple***	1
21	***Feature 20: Payment Gateway Integration, Voucher & Checkout***		***13***
21.1	***Checkout***	***Medium***	2
21.2	***Order Summary***	***Simple***	1
21.3	***Organizer-specific Voucher***	***Medium***	2
21.4	***System-wide Voucher***	***Medium***	2
21.5	***Real-time Voucher Validation***	***Medium***	2
21.6	***Calculate Discounted Amount***	***Medium***	1
21.7	***Payment Gateway Integration***	***Complex***	2
21.8	***Payment Webhook Handling***	***Complex***	1
22	***Feature 21: System Health & Observability Dashboard***		***8***
22.1	***Monitor API Response Time***	***Medium***	1
22.2	***Monitor Error Rate***	***Medium***	1
22.3	***Monitor Microservice Status***	***Medium***	2
22.4	***Monitor Virtual Queue Traffic***	***Medium***	1
22.5	***Prometheus Integration***	***Medium***	1
22.6	***Grafana Dashboard***	***Medium***	2
		Total Estimated Effort (man-days)	***250***

1.2 Project Objectives

The objective of the **ConcertShield System** project is to design and implement a secure event ticketing and management platform for music-related events. The system supports end-to-end event ticketing operations, including user authentication and account management, eKYC and identity verification, role-based access control, event discovery and search, virtual queue management, interactive seating and ticket selection, digital ticket wallet and dynamic QR codes, ticket return and re-issue, event creation and lifecycle management, seating chart and ticket configuration, revenue analytics, real-time check-in monitoring, QR code-based ticket validation, incident reporting, event approval and dispute resolution, AI-based risk scoring and fraud detection, system parameter configuration, audit logging, real-time notifications, payment and voucher management, and system health monitoring.

The project aims to improve the ticket purchasing experience, ensure fair access to high-demand events, reduce ticket fraud and scalping risks, prevent double-booking, improve attendee check-in operations, and provide Event Organizers, Staff, and Administrators with reliable real-time management and monitoring tools.

Target metrics (Quality – Time – Cost)

#	Testing Stage	Target / Metric	No. of Defects	% of Defect	Notes
1	Reviewing	100% of key requirements, diagrams, and documents reviewed before submission; all major documents checked by at least one reviewer	≤ 2 document issues after review	≤ 5%	Use checklist, peer review, and supervisor feedback.
2	Unit Test	UT coverage ≥ 85% for core business logic, authentication, ticket booking, seat management, payment, notification, and fraud-related functions.	≤ 8 minor defects	≤ 10%	Focus on critical business rules and security-sensitive functions.

3	System Test	100% of major features executed in the staging environment.	≤ 8 unresolved defects	≤ 15%	End-to-end validation by Customer, Event Organizer, Staff, and Admin roles.
4	Acceptance Test	100% of approved UAT scenarios passed or accepted with documented limitations.	0 critical defects	0%	UAT with supervisor and final defect confirmation.

Milestone Timeliness (%): ≥ 95% milestones delivered on or before their planned due dates.

Allocated Effort (man-days): 5 members × 13 weeks × 5 days per week ≈ 325 man-days

- **Requirements analysis:** 50 man-days (15.4%)
- **System and database design:** 50 man-days (15.4%)
- **Coding and implementation:** 176 man-days (54.2%)
- **Testing:** 30 man-days (9.2%)
- **Project management and documentation:** 19 man-days (5.8%)

Total: 325 man-days (100%)

1.3 Project Risks

#	Risk Description	Impact	Possibility	Response Plans
1	A team member withdraws from the project or becomes unable to continue.	HIGH	LOW	Clearly distribute responsibilities, maintain source code on Git, document each module, and ensure knowledge sharing among team members.
2	A development computer or other hardware fails.	HIGH	LOW	Regularly back up source code, database, and configuration files using Git and cloud storage, and prepare a backup development environment.
3	Team members lack experience	HIGH	MEDIUM	Conduct early technical research, divide complex tasks into smaller tasks,

	with technologies such as SignalR, AI, eKYC, or distributed locking.			create Proof of Concept (PoC) implementations, and support knowledge sharing within the team.
4	AI Risk Scoring and Fraud Detection do not achieve the expected accuracy.	HIGH	MEDIUM	Limit AI usage to risk-assessment support, provide rule-based validation as a fallback, and test the AI using predefined normal and abnormal scenarios.
5	eKYC functions such as OCR, Face Matching, or Liveness Detection are unstable.	HIGH	MEDIUM	Test each component independently, prepare fallback flows, and limit the impact of eKYC failures on the overall ticket purchasing process.
6	Payment Gateway or Webhook integration fails or behaves unexpectedly.	HIGH	MEDIUM	Use a sandbox environment for testing, separate payment processing from booking logic, implement timeout/retry handling, and validate webhooks before updating transaction status.
7	Race conditions occur in the Virtual Queue or real-time seat synchronization.	HIGH	MEDIUM	Define clear state transitions, apply distributed locking and transaction control, and perform concurrent request and stress testing.
8	Double-booking occurs when multiple users attempt to select the same seat simultaneously.	HIGH	LOW	Validate seat availability on the backend, apply appropriate locking mechanisms, and perform concurrency testing before release.
9	Scope creep occurs because of additional or changing requirements.	HIGH	HIGH	Clearly define the project scope, prioritize mandatory features, document new requests, and only accept changes that do not significantly affect major milestones.

10	Development progress falls behind the 13-week project schedule.	HIGH	MEDIUM	Monitor progress weekly, prioritize critical features, divide work into milestones, and reallocate tasks among team members when necessary.

2. Management Approach

To ensure the successful execution of the **ConcertShield System (CSS)** project within the capstone timeline, the team adopts a structured, modern management approach. This approach balances agile software development, comprehensive quality assurance, continuous team skill enhancement, transparent project governance, and controlled configuration management.

Key pillars of our project management approach include:

- **Agile-Scrum Process:** Implementing 2-week Sprint iterations to continuously design, develop, test, and deliver core system features (eKYC, Virtual Queue, Redis Distributed Locking, AI Fraud Detection) while accommodating feedback from the project supervisor.
- **Quality Governance:** Combining Defect Prevention (coding standards, static analysis), Peer Reviews (GitHub Pull Requests), and multi-level automated/manual testing (Unit, Integration, k6 Load Testing) to guarantee high system stability and security.
- **Targeted Capability Building:** Conducting structured internal training sessions before and during execution to ensure all team members possess the technical competencies required for complex backend patterns, frontend state management, and load testing.
- **Structured Communication & CM:** Maintaining clear weekly progress reports with the supervisor, establishing defined roles (Product Owner, Scrum Master, Developers), and controlling documents and source codes using standard Version Control tools (GitHub, Google Drive).

2.1 Project Process

Software Development Process Model: Scrum (Agile Framework)

The **Concert Shield System (CSS)** team applies the **Scrum** framework—an iterative and incremental Agile software development methodology—to manage and execute the project.

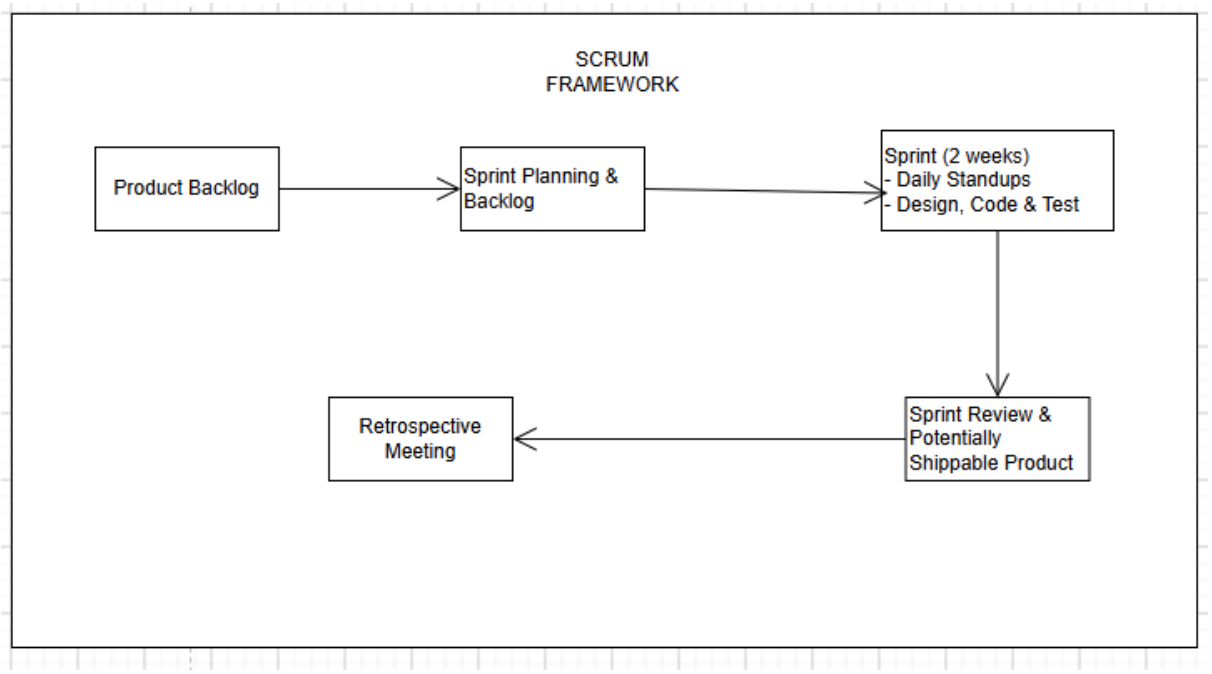


Figure 2.1: Scrum Software Development Process Model

Definition

Scrum is an Agile framework designed for managing complex software development projects through short, fixed-length iterations called **Sprints** (typically 2 weeks). Work is prioritized in a **Product Backlog** and refined into smaller, actionable tasks delivered incrementally.

Characteristics

- **Iterative & Incremental Delivery:** The software is developed in small, manageable increments (Sprints), delivering working features at the end of each iteration.
- **Core Roles:** Product Owner (prioritizes requirements), Scrum Master (facilitates process), and Development Team (cross-functional execution).
- **Key Ceremonies:** Sprint Planning, Daily Standup (15-minute daily sync), Sprint Review (demo to stakeholders), and Sprint Retrospective (process improvement).

Strengths

- High adaptability to changing requirements or continuous technical feedback.
- Early risk identification and transparent progress tracking across the team.
- continuous integration and regular delivery of working software increments.

Weaknesses

- Requires high discipline and active participation from all team members.
- Risk of scope creep if backlog items are not strictly managed.

Reasons for Selection

- **Handling High Complexity & Technical Integrations:** ESS involves complex integrations including high-concurrency booking engines (.NET 8, Redis Distributed

Lock, Saga/Outbox patterns), eKYC (OCR, Liveness Detection), and AI Risk Scoring (Python FastAPI). Scrum allows the team to break down these modules into manageable Sprints.

- **Frequent Feedback Loops:** Weekly supervisor reviews and milestone submissions require continuous refinement of requirements and documentation.

2.2 Quality Management

To reach the project quality objectives and ensure system stability, security, and high performance, the team enforces the following quality assurance approaches:

- **Defect Prevention:** Proactive practices applied before and during implementation to eliminate defects early. Includes setting strict coding standards, performing static code analysis (SonarQube/Linters), using TypeScript/strongly-typed models, establishing automated CI/CD pipelines, and writing detailed functional specifications prior to coding.
- **Reviewing:** Structured, peer-based evaluation of project deliverables to detect errors early. Code reviews are enforced via Pull Requests (PRs) on GitHub, requiring approval from at least one peer before merging to the main branch. Documentation reviews (SRS, Architecture, WBS) are conducted prior to submission.
- **Unit Testing:** Automated testing of individual, isolated software components (functions, services, controllers) to verify correctness. The backend (.NET 8) uses xUnit/Moq to test business logic and distributed locking mechanisms, while the AI service (FastAPI) utilizes PyTest. A minimum target unit test coverage is enforced.
- **System Testing:** Black-box end-to-end testing of the fully integrated application to validate compliance with functional and non-functional requirements. Covers end-to-end workflows (eKYC verification, Virtual Queue, seat selection, VNPay payment, Dynamic QR issuance), along with load testing using k6/JMeter to simulate high-concurrency flash sales.

2.3 Training Plan

Training Area	Participants	When, Duration	Waiver Criteria
ReactJS & Dynamic State Management	All Team Members	23/08/2026, 02 weeks	Mandatory
.NET 8	All Team Members	23/08/2026, 02 weeks	Mandatory
Python FastAPI & AI Integration	All Team Members	30/08/2026, 01 week	Mandatory

Redis Distributed Locking & High Concurrency	All Team Members	30/08/2026, 01 week	Mandatory
System Load Testing (k6 / JMeter)	All Team Members	06/09/2026, 01 week	Mandatory

3. Project Deliverables

#	Deliverable	Due Date	Notes
1	Report 1 – Project Introduction & Proposal	dd/MM/yyyy	Project proposal, product background, existing systems analysis, and business opportunity.
2	Report 2 - Project Management Plan	...	Project team allocation, project scope definition, and limitation boundaries.
3	Report 3 – Software Requirements Specification (SRS)	...	Functional requirements, actor roles (Customer, Organizer, Staff, Admin), use cases, business rules.
4	Report 4 – Software Design Specification (SDS/SDD)	...	System architecture (.NET 8, ReactJS, FastAPI), database design, and concurrency patterns.
5	System Implementation (Coding & Testing)	...	Backend/frontend coding, AI service integration, and payment gateway integration.
6	Report 5 – Test Documentation & Iteration Results	...	Test cases, functional testing results, and load testing validation.

8	Report 6 – User Guides & Deployment Documentation		User manuals for system actors and system deployment instructions.
8	Report 7 + Final Thesis + Slide + Demo + Supervisor Review		Final thesis submission, product demo, and supervisor approval
9	Submit Thesis to Grading Council		Forwarding the thesis and grading rubrics to the review council.
10	Capstone Final Defense		Official presentation and product defense before the council.

4. Responsibility Assignments

D~Do; R~Review; S~Support; I~Informed; <blank>- Omitted

Responsibility	**Tanmnce182352**	**Khiemnpgce182252**	**Vinhntc181630**	**Tainhce182569**	**Minhthce180705**
Project Planning & Tracking	R	D	D	S	S
Prepare Report 1 – Project Introduction	R	D	D	S	S
Prepare Report 2 – Project Management Plan	R	D	S	D	S

Prepare Report 3 – Software Requirements Specification	S	D	D	R	S
Prepare Report 4 – Software Design Document	D	D	S	R	S
Prepare Report 5 – Test Documentation	S	S	R	D	D
Prepare Report 6 – User Guides	S	S	D	D	R
Prepare Report 7 – Final Project Report	R	D	D	D	S
Frontend Development (ReactJS)	S	S	R	D	D
Backend Development (.NET 8)	R	Đ	D	S	S
AI Module Development (FastAPI)	D	R	D	S	S
Database Design (PostgreSQL)	D	D	S	R	S
Payment Gateway Integration	S	R	S	D	D
Testing & Defect Tracking	S	R	S	D	D
Deployment & Environment Setup	R	D	S	D	S
Final Report & Documentation	R	D	D	D	S

5. Project Communications

Communication Item	**Who/ Target**	**Purpose**	**When, Frequency**	**Type, Tool, Method(s)**
Daily Online Check-in	All team members	Report daily progress, blockers, and next tasks.	Daily	Zalo
Weekly Team	All team	Review sprint progress,	Every Sunday	Google Meet

Meeting	members	confirm completed tasks, assign next sprint work, and update risks.		
Supervisor Meeting	All team members, Supervisor	Report milestone progress, get feedback, clarify scope and documentation issues.	Weekly or bi-weekly depending on supervisor schedule	Direct meeting / Google Meet
Document Review Session	All team members	Review report sections before milestone submission.	Before each report submission	Google Docs comments, checklist, Google Meet
Technical Review Session	In-charge members, Leader	Review architecture decisions, API contracts, implementation blockers, and integration issues.	At least once per sprint	Google Meet, GitHub Pull Requests
Documentation Update	Team	Share latest documents	Weekly	Google Drive

6. Configuration Management

6.1 Document Management

All documents of the **ConcertShield System (CSS)** project — including the Software Requirements Specification (SRS), microservices architecture design documents, capstone reports (Report 1, 2, 3...), meeting minutes, and tracking sheets — are managed centrally on **Google Drive**, using **Google Docs** and **Google Sheets** for drafting and collaboration.

Introduction to the tool: Google Drive is a cloud-based storage and collaboration platform that allows all members of group SEP490-13 (Tân, Vinh, Khiêm, Tài, Minh) to co-edit documents in real time, with automatic version history tracking.

Strengths

- Real-time collaboration: all five members can contribute simultaneously (as reflected in the Record of Changes table in Report 1, where each member owns a section – Tân wrote the Project Introduction, Khiêm wrote Major Features, Tài wrote Business Opportunity & Vision, Minh wrote Existing Systems).
- Built-in Version History makes it easy to track who changed what and when, supporting the "Record of Changes" table (Added/Modified/Deleted) used in each report.
- Easy to share with the supervisor (Ms. Luong Hoang Huong) under Viewer/Commenter access without granting edit rights.

Weaknesses

- Not well suited for complex technical artifacts (such as microservices architecture diagrams or ERDs) — needs to be combined with tools like draw.io or Figma embedded into Docs.
- Version control is not as strict as Git when multiple members edit the same section at the same time.

How it is managed

- The Drive folder structure is organized by document type: Reports (Report 1 – Project Introduction, Report 2...), Design, Meeting Notes, Tracking.
- Each major report follows a version naming convention (v1.0, v1.1...) whenever a major change is approved by the team; minor edits rely on the built-in Version History.
- The **Record of Changes** table at the start of each report (Date – A/M/D – In charge – Description) is updated immediately whenever an edit is made, making it easy for the supervisor to track each member's contribution.
- Before submitting each milestone (Report 1, 2, 3...), the whole team cross-reviews the content against the grading rubric before finalizing the submitted version.



Figure 3: Google Docs

6.2 Source Code Management

Source code for **ConcertShield System** is managed using **GitHub**, which fits the project's **microservices architecture on .NET 8** (services such as IAM, Event & Seat, Booking & Queue, Compliance & Resale, Payment & Refund, and Ticketing & Check-in).

Introduction to the tool: GitHub is a Git-based source code hosting platform that supports branching, Pull Requests/Code Review, and CI/CD integration (GitHub Actions) — well suited for five members developing multiple microservices in parallel without stepping on each other's work.

Strengths

- Supports clean module separation: each member can own a specific service (e.g., Khiêm handles Booking & Queue with Redis Distributed Lock, …)
- Mandatory Pull Request review helps ensure code quality before merging into the shared branch — critical since the system deals with high-concurrency logic (Virtual Queue, Saga/Outbox pattern) where small bugs can easily cause race conditions.
- GitHub Actions automatically runs unit tests per service, supporting isolated testing before integration and load testing (k6/JMeter).
- GitHub Issues linked to each feature (FE-01 through FE-21 in Report 1) make it easy to trace which feature is being worked on, by whom, and its current status.

Weaknesses

- With a multi-service architecture (.NET backend, React frontend, FastAPI AI service), clear repository organization is required — otherwise dependencies between services (e.g., the Booking service calling the AI Risk Scoring service) can become confusing.
- Merge conflicts can occur when multiple members edit shared libraries at the same time (e.g., DTOs shared across microservices).

Reason for choosing this tool: GitHub was chosen because it is free, integrates well with CI/CD, fits a multi-repository multi-module microservices model, and enforces code review — essential for a system with strict correctness requirements under concurrent load.

How it is managed

- Repositories are organized by component: ess-backend (the .NET 8 microservices: IAM, Event & Seat, Booking & Queue, Compliance & Resale, Payment & Refund, Ticketing & Check-in), ess-frontend (ReactJS), ess-ai-service (Python FastAPI — AI Risk Scoring, Bot Detection).
- Branch structure:
    1. **main**: stable code, ready for deployment/demo.
    2. **develop**: integration branch where services are merged and tested together before reaching main.

3. **feature/xxx**: each member creates a dedicated branch per assigned feature (e.g., feature/virtual-queue, feature/dynamic-qr, feature/ekyc-flow).

- Commit message convention: feat(booking): implement redis distributed lock, fix(payment): handle VNPay webhook retry, docs(ai-service): update risk scoring readme.
- Workflow:
    1. A member creates a feature/xxx branch from develop and codes the assigned feature (FE-xx) as defined in Report 1.
    2. The member pushes the code and opens a Pull Request into develop.
    3. GitHub Actions automatically runs unit tests for the relevant service.
    4. At least one other member reviews the PR (checking logic, especially anything related to Redis locking/Saga pattern, to avoid race conditions).
    5. The team leader (Tân) performs the final review and decides whether to merge or request changes.
    6. Once development is stable after integration and load testing (k6/JMeter), a PR is opened from develop into main to prepare for release/demo.
    7. GitHub Issues track each feature (FE-01 → FE-21) with labels bug, feature, in-progress, done, linked directly to the PR that resolves it.



Figure 4: GitHub

6.3 Tools & Infrastructures

Category	**Tools / Infrastructure**
Technology	React.js, Next.js (FrontEnd), .NET 8 (BackEnd)
Database	PostgreSQL
IDEs/Editors	Visual Studio Code, IntelliJ IDEA, Visual Studio
Diagramming	Draw.io, StarUML, PlantUML, Lucidchart, Sequencediagram

Documentation	Microsoft Word, Google Docs, Google Sheets
Version Control	GitHub (Source Code), Google Drive (Documents)
Deployment server	Vercel
Project management	Google Sheets

1/ Tài liệu tham khảo ?

2/ Cite tài liệu tham khảo vào nội dung